

Resolução dos Exercícios

Capítulo: Laços

1. Indique quais das seguintes afirmações são Falsas e quais são Verdadeiras:

- 1.1 A condição dentro de um laço `while` e `do...while` tem que ser colocada sempre dentro de parênteses. - **verdadeira**
- 1.2 Os laços `while` e `for` executam **SEMPRE**, pelo menos uma vez, o corpo do laço. - **falsa**
- 1.3 O laço `do...while` executa **SEMPRE**, pelo menos uma vez, o corpo do laço. - **verdadeira**
- 1.4 As três componentes, dentro de parênteses, do laço `for` são todas obrigatórias. - **falsa**
- 1.5 As três componentes, dentro de parênteses, do laço `for` são todas facultativas. - **verdadeira**
- 1.6 Quando em um laço `for` se tem que realizar mais do que uma carga inicial ou mais do que um incremento, as diversas instruções, em cada uma das componentes, devem ser separadas por vírgulas e não ponto-e-vírgula, de forma a manter o formato do laço `for`. - **verdadeira**
- 1.7 A instrução de um laço é sempre executada. - **falsa**
- 1.8 No laço `for`, o número de vezes que as cargas iniciais são executadas é sempre igual ao número de iterações do laço. - **falsa**
- 1.9 No laço `for` ou `while`, o número de vezes que a condição é testada é sempre igual ao número de iterações do laço. - **falsa**
- 1.10 No laço `for`, o número de vezes que a instrução é executada é sempre igual ao número de vezes que é executada a pós-instrução. - **falsa**
- 1.11 Um laço, quando está dentro de outro laço, necessita de chaves. - **falsa**
- 1.12 Não há limite para o número de laços dentro de outros laços. - **verdadeira**
- 1.13 No laço `do...while`, a instrução é sempre executada pelo menos uma vez. - **verdadeira**
- 1.14 Em todos os laços, a condição é sempre testada pelo menos uma vez. -

verdadeira

1.15 Os laços `while` e `do...while` incrementam automaticamente a variável de controle. - **falsa**

1.16 Os laços são executados até que a condição escrita se verifique - **falsa**

1.17 O laço `do...while` aparece escrito menos frequentemente do que qualquer dos outros. - **verdadeira**

1.18 Um `break`, quando presente dentro de um laço, termina o programa. - **falsa**

1.19 No laço `while`, antes de se executar a instrução `continue`, deve-se atualizar a variável de controle do laço. - **verdadeira**

1.20 Qualquer dos laços pode ser sempre escrito a partir do laço `while`. - **verdadeira**

2. Escreva genericamente o laço `do...while` como um laço `while`.

```
instruções;

while (condição) {
    instruções;
}
```

3. Escreva genericamente o laço `for` como um laço `while`.

```
inicialização;

while (condição) {
    instruções;
    incremento;
}
```

4. Escreva genericamente o laço `while` como um laço `for`.

```
for ( ; condição; ) {  
    instruções;  
}
```

5. Qual a diferença de execução da instrução `break` quando presente em um `for` e em um `while`?

Não existe diferença. Ambos os laços são encerrados, o programa segue para a próxima instrução.

6. Qual a diferença de execução da instrução `continue` quando presente em um `for` e em um `while`?

No `for`, após a instrução `continue`, é executada a instrução de incremento ou decremento; depois disso, segue-se para o teste de condição. No `while`, depois da instrução `continue`, é realizado o teste de condição diretamente. O incremento ou decremento deve ser feito antes da instrução `continue` para evitar um loop infinito.

7. Reescreva o seguinte trecho de um programa, utilizando o laço `while`:

```
for (i = 1; i <= 20; i++)  
    if (i == 10)  
        continue;  
    else  
        printf("%d\n", i);
```

```
i = 1  
while (i <= 20) {  
    if (i == 10) {  
        i++;  
    }  
    printf("%d\n", i);  
}
```

```

        continue;
    }
    else {
        printf("%d\n", i);
        i++;
    }
}

```

8. Exercício de exame

Escreva um programa que coloque na tela meia árvore de natal com asteriscos. O número de ramos deverá ser introduzido pelo usuário. Exemplos com 3, 4 e 5 ramos:

```

*           *           *
* *        * *        * *
* * *      * * *      * * *
           * * * *      * * * *
                           * * * * *

```

[Ver código-fonte \(exe08.c\)](#)

9. Exercício de exame

Altere o programa anterior de forma que, em vez de asteriscos, sejam escritas letras em cada nível, começando o nível inicial com a letra 'A'.. Exemplos com 3, 4 e 5 ramos:

```

A           A           A
B B        B B        B B
C C C      C C C      C C C
           D D D D      D D D D
                           E E E E E

```

[Ver código-fonte \(exe09.c\)](#)

10. Escreva um programa que solicite um número ao usuário até que o valor deste esteja entre os valores 1 e 100.

[Ver código-fonte \(exe10.c\)](#)

11. Exercício de exame

Escreva um programa em C que escreva na tela toda a tabela ASCII(0.255 chars), escrevendo em cada linha de código ASCII e o caractere correspondente.

Exemplo:

```
...  
65 --> A  
66 --> B  
67 --> C  
...
```

A fim de que o usuário possa ver todos os caracteres escritos, a tela deve ser parada de 20 em 20 linhas, até que o usuário pressione a tecla 'c' ou 'C' seguida de <ENTER> para continuar a mostrar os próximos 20 caracteres (20 linhas).

[Ver código-fonte \(exe11.c\)](#)

12. Escreva um programa completo em C que solicite ao usuário dois números inteiros entre 0 e 255 e escreva na tela todos os caracteres da tabela ASCII cujos códigos variem entre os dois números introduzidos, escrevendo em cada linha o código ASCII e o caractere correspondente.

Exemplos:

```
Introd. dois N°: 65 120
```

```
65 --> A
```

```
66 --> B
```

```
67 --> C
```

```
Introd. dois N°: 120 65
```

```
65 --> A
```

```
66 --> B
```

```
67 --> C
```

[Ver código-fonte \(exe12.c\)](#)

13. Exercício de Exame

Qual a diferença (se existir) entre as duas seguintes instruções: `x = ++i` e `x = i++`? Dê exemplos.

`x = ++i`: Primeiro, o valor de `i` é incrementado; em seguida, o novo valor é atribuído a `x`. Ao final da operação, ambas as variáveis terão o mesmo valor.

`x = i++`: Primeiro, o valor atual de `i` é atribuído a `x`; somente depois disso o valor de `i` é incrementado. Ao final, a variável `i` terá uma unidade a mais que a variável `x`.

14. Se uma variável `x` se encontra isolada, `++x` e `x++` são equivalentes?

Sim, as instruções são equivalentes nesse contexto. Em ambas, o valor de `x` será acrescido de uma unidade ao final da execução.

15. Pode-se utilizar os operadores `++` e `--` em constantes?

Não, pois os valores das constantes são imutáveis e não permitem reatribuição.

16. Qual a diferença entre `a=-2` e `a-=2`?

`a = -2` O valor negativo -2 é atribuído à variável `a`. Independentemente do valor anterior, `a` passará a valer -2.

`a -= 2` O valor atual da variável `a` é subtraído em 2 unidades. É uma forma abreviada de escrever `a = a - 2`.

17. Qual o resultado do seguinte programa:

```
n=0;
do
    { printf(" ...");
      n=n+1;
    }
while (n!=0);
```

No código apresentado, a mensagem `" ..."` será exibida infinitamente. Isso ocorre porque a variável `x` inicia em 0, mas sofre um incremento dentro do bloco `do`, passando a valer 1 antes de chegar ao teste de condição. Como a condição do `while` é `n != 0`, e o valor de `n` continuará crescendo, ela sempre será verdadeira, impedindo o encerramento do laço enquanto houver recursos computacionais disponíveis.

18. Exercício de exame

Qual a diferença entre os dois trechos de código:

```
i = 0;
while (i++)
    printf("%d\n", i);
```

```
i = 0;
while (++i)
    printf("%d\n", i);
```

No primeiro código, quando a condição do `while` for testada, o valor de `i` será 0. Depois do teste, `i` será incrementado; portanto, o código dentro do `while` não será executado. No segundo, o valor de `i` será incrementado antes do teste do `while`. Quando o teste for feito, o valor de `i` será 1, e o código dentro do `while` será executado.

19. O que faz o seguinte código:

```
for (i=1; i<=200; i++);  
    printf("%d\n", i);
```

O código inicializa a variável `i` com o valor 1 e a incrementa enquanto ela for menor ou igual a 200. Devido ao ponto e vírgula (`;`) imediatamente após o comando `for`, o laço é executado vazio, sem repetir o `printf`. Quando `i` atinge o valor 201, a condição de parada é satisfeita e o laço termina. Somente então o comando `printf` é executado uma única vez, exibindo o valor final de `i`, que é 201.

20. Quantas vezes são executadas as instruções dos seguintes laços:

- 20.1 `for (i=1 ; i<=20 ; i++) ...`
- 20.2 `for (i=1 ; i<=20 ; i+=2) ...`
- 20.3 `for (i= -20 ; i<=20 ; i++) ...`
- 20.4 `for (i=1 ; i<=10 ; i++)`
- 20.5 `for (j=1 ; j<=5 ; j++) ...`

- 20.1 = 20 vezes
- 20.2 = 10 vezes
- 20.3 = 41 vezes
- 20.4 = 10 vezes
- 20.5 = 5 vezes

21. Exercício de exame

Preencha os espaços em branco com o valor das respectivas colunas, depois de executada a instrução à sua esquerda.

Nota: Devem ser utilizados os valores alterados das variáveis em cada uma das linhas.

	x	y	z
	5	10	7
x++	6	10	7
y *= (z++ + 2)	6	90	8
y = !x	6	0	8
z = (x % 2) + --y	6	-1	-1
z = (x == y)	6	-1	0
!z	6	-1	0
z = x % 8	6	-1	6